KDL OS: Project Walkthrough

Kalam Dream Labs
November 7, 2026

CONTENTS

KDL OS: Project Walkthrough

Part 1. What you are installing

Part 2. Install the tools

Part 3. Step 1: Clone the project

Part 4. Step 2: Create the settings file (.env)

Part 5. Step 3 to 5: Start, seed, log in

- Step 3. Start every service
- Step 4. Create the admin user (first run only)
- Step 5. Log in
- Forgot the admin password, or locked out
- Service addresses

Part 6. Developer route: run without Docker for the apps

- 6.1 Start infrastructure only
- 6.2 Backend
- 6.3 Frontend
- 6.4 AI services
- 6.5 Daily commands

Part 7. Check that the project is healthy

CONFIDENTIAL

Part 8. Tour of the project

8.1 How the backend is built

8.2 The modules

8.3 Roles

8.4 The frontend

Part 9. Build a website with the Template Engine

Part 10. Windows setup (once, before Part 3)

Part 11. Production run

Part 12. Troubleshooting

Part 13. Contributing

Part 14. Where to read next

CONFIDENTIAL

KDL OS: Project Walkthrough

A step-by-step guide to the whole project. Start from an empty computer, clone the code, install, set up, log in, build a website, and learn how the code is laid out.

CONFIDENTIAL

Part 1. What you are installing

KDL OS (Kalam Dream Labs Starter Kit) is a modular SaaS foundation. One installation can run an Admin app, a Website (CMS, menus, Section Builder, themes, SEO), Landing Pages, forms and media. Modules are switched on from the admin panel, so building a project needs no source edits.

Layer	Technology
Backend	Node.js 20, Express 5, Prisma 6
Database	PostgreSQL 15
Cache and queue	Redis 7, BullMQ
Frontend	Next.js 15 (App Router), TypeScript 5
UI	TailwindCSS 3, shadcn/ui, Zustand 5, TanStack Query 5
AI services	LangChain.js, ChromaDB, OpenRouter API
File storage	MinIO
Search	MeiliSearch
Proxy	Nginx
Package manager	pnpm (frontend), npm (backend, ai-services)

How the pieces talk to each other:

```
Browser
   |
 Nginx :80
   |-- /api  -->  Backend (Express) :4000  -->  PostgreSQL, Redis, MinIO,
MeiliSearch
   |-- /ai   -->  AI Services :5000        -->  ChromaDB, OpenRouter
   |-- /     -->  Frontend (Next.js) :3000
```

Always open the app through Nginx at `http://localhost`. It is the single entry point.

CONFIDENTIAL

Part 2. Install the tools

You need three tools for the Docker route (the recommended one). You need two more for the developer route in Part 6.

Tool	Get it from	Check
Git	https://git-scm.comSMARTPANTS_PRESERVED_270SMARTPANTS_PRESERVED_271	`git --version`
Docker Desktop	https://www.docker.com/products/docker-desktopSMARTPANTS_PRESERVED_280SMARTPANTS_PRESERVED_281	`docker --version`
OpenSSL	Already on Mac and Linux	`openssl version`
Node.js 20 (developer route only)	https://nodejs.orgSMARTPANTS_PRESERVED_298SMARTPANTS_PRESERVED_299	`node -v`
pnpm (developer route only)	`npm install -g pnpm`	`pnpm -v`

Docker Desktop settings: open Settings, Resources, Memory, and set it to 8 GB. The first build runs out of memory with less.

Windows users: do the Windows setup in Part 10 first, then type every command in the Ubuntu window.

CONFIDENTIAL

Part 3. Step 1: Clone the project

```
git clone https://github.com/F9info/kdl-os.git
cd kdl-os
```

Stay inside this folder for every command that follows.

Useful to know: the repo has more than one remote on some machines. `git remote -v` shows them. Work on a branch, never on `master`:

```
git checkout -b my-feature
```


CONFIDENTIAL

Part 4. Step 2: Create the settings file (`.env`)

The `.env` file holds every secret and port. It is never committed. The block below creates it with fresh random passwords. Copy the whole block, paste it into the terminal, press Enter.

```
PG_PASS=$(openssl rand -hex 16)
REDIS_PASS=$(openssl rand -hex 16)
MINIO_PASS=$(openssl rand -hex 16)

cat > .env <<EOF
NODE_ENV=development
APP_PORT=4000
FRONTEND_URL=http://localhost
BACKEND_URL=http://localhost:4000
AI_SERVICES_URL=http://localhost:5000
CORS_ORIGIN=http://localhost,http://localhost:3000,http://localhost:3001
JWT_SECRET=$(openssl rand -hex 32)
JWT_REFRESH_SECRET=$(openssl rand -hex 32)
JWT_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d
APP_ENCRYPTION_KEY=$(openssl rand -hex 32)
SEED_ADMIN_EMAIL=admin@kdl.com
SEED_ADMIN_PASSWORD=$(openssl rand -hex 16)
POSTGRES_USER=postgres
POSTGRES_PASSWORD=$PG_PASS
POSTGRES_DB=kdl_db
DATABASE_URL=postgresql://postgres:$PG_PASS@localhost:5433/kdl_db
REDIS_PASSWORD=$REDIS_PASS
REDIS_URL=redis://:$REDIS_PASS@localhost:6380
MINIO_ROOT_USER=kdl_minio
MINIO_ROOT_PASSWORD=$MINIO_PASS
MINIO_ACCESS_KEY=kdl_minio
MINIO_SECRET_KEY=$MINIO_PASS
MINIO_ENDPOINT=localhost
MINIO_PORT=9000
MINIO_USE_SSL=false
MINIO_BUCKET=kdl-media
MEILISEARCH_HOST=http://localhost:7700
MEILI_MASTER_KEY=$(openssl rand -hex 24)
MEILISEARCH_API_KEY=replace-after-first-boot
MEILI_SEARCH_API_KEY=replace-after-first-boot
CHROMADB_URL=http://localhost:8000
ANTHROPIC_API_KEY=local
OPENROUTER_API_KEY=local
EOF
chmod 600 .env
```


CONFIDENTIAL

Check it worked: `ls -la .env` shows a file of about 2 KB.

Notes:

- No accounts or API keys are needed to run the core product. `ANTHROPIC_API_KEY` and `OPENROUTER_API_KEY` are placeholders. Put real keys there only when you want AI features.
- `SEED_ADMIN_PASSWORD` is a random value, so no one else knows your admin password. It must be 12 or more characters.
- `MEILISEARCH_API_KEY` and `MEILI_SEARCH_API_KEY` are placeholders. After first boot, create scoped keys through the MeiliSearch `/keys` API with the master key (see the comments in `.env.example`) and put them here.
- Never share or upload `.env`.
- Prefer a template? `cp .env.example .env`, then replace every `change_this_...` value. `docs/ENV_REFERENCE.md` explains each variable.

CONFIDENTIAL

Part 5. Step 3 to 5: Start, seed, log in

Step 3. Start every service

```
docker compose up -d --build
```

The first run takes 10 to 20 minutes. Later starts take seconds. The backend applies database migrations on start.

Check status:

```
docker compose ps
```

Wait until `backend` shows `healthy` (about a minute after the build ends).

Step 4. Create the admin user (first run only)

```
docker compose exec backend node prisma/seed.js
```

Expected output: `Seeded admin: admin@kdl.com`.

Step 5. Log in

1. Open http://localhost/loginSMARTPANTS_PRESERVED_362SMARTPANTS_PRESERVED_363
2. Email: `admin@kdl.com`
3. Password: `Admin@123`. This is a development password for your own computer only. Never use it on a server anyone else can reach. Change it when KDL OS asks (next step).
   - On a brand-new database the seed cannot create this password (it needs 12 or more characters). If the login fails, use the `SEED_ADMIN_PASSWORD` value from your `.env` (`grep SEED_ADMIN_PASSWORD .env`). If you removed that line, the Docker stack default is `kdl-dev-seed-password`.
4. KDL OS forces you to choose a new password. This is expected, not a bug.

CONFIDENTIAL

After login the Setup Wizard asks for company identity and which products to turn on (Website, Landing Pages). If it asks “Do you need a front-end website?”, choose Yes to get a starter site with Home, About, Services, Contact and FAQ.

Forgot the admin password, or locked out

```
NEWPASS=$(openssl rand -base64 18)
echo "$NEWPASS"
docker compose exec backend node scripts/reset-admin-password.js --email
admin@kdl.com --password "$NEWPASS"
```

Then sign in with the password the first command printed, or replace `"$NEWPASS"` with a password of your own. Outside production the password needs 8 or more characters. With `NODE_ENV=production` it needs 12 or more and the script needs `--force`. Do not reuse a short, well-known password such as the examples in older docs. Re-running the seed does not reset an existing admin, so use this script.

”Account locked” after many failed sign-ins: wait one minute. During heavy testing a 429 on login clears with `docker compose restart backend`.

CONFIDENTIAL

Service addresses

Service	Address	Notes
App through Nginx	http://localhostSMARTPANTS_PRESERVED_402SMART-PANTS_PRESERVED_403	Use this one
Frontend direct	http://localhost:3001SMARTPANTS_PRESERVED_412S-MARTPANTS_PRESERVED_413	Next.js
Backend API	http://localhost:4000SMARTPANTS_PRESERVED_422S-MARTPANTS_PRESERVED_423	REST, health at `/health`
AI services	http://localhost:5001SMARTPANTS_PRESERVED_432S-MARTPANTS_PRESERVED_433	LLM layer
MinIO API / console	http://localhost:9002SMARTPANTS_PRESERVED_442 / 9003	Object storage
MeiliSearch	http://localhost:7700SMARTPANTS_PRESERVED_452S-MARTPANTS_PRESERVED_453	Search dashboard
ChromaDB	http://localhost:8000SMARTPANTS_PRESERVED_462S-MARTPANTS_PRESERVED_463	Vector store
PostgreSQL	localhost:5433	Not the default 5432
Redis	localhost:6380	Not the default 6379

All of these ports are for your own computer only. The development compose file binds them to 127.0.0.1. Do not open them on a firewall, router or cloud host.

Dev logins: PostgreSQL `postgres` with the password in your `.env`. MinIO uses the `MINIO_ROOT_USER` and `MINIO_ROOT_PASSWORD` you set. Change every password before production.

CONFIDENTIAL

Part 6. Developer route: run without Docker for the apps

Use this when you edit code and want hot reload. Infrastructure still runs in Docker.

6.1 Start infrastructure only

```
docker compose -f docker-compose.infra.yml up -d
docker compose -f docker-compose.infra.yml ps
```

PostgreSQL, Redis, MinIO, MeiliSearch and ChromaDB run here. All should show `healthy` or `running`.

6.2 Backend

```
cd backend
npm install
mkdir -p logs
npm run db:migrate
npm run db:seed
npm run dev
```

`npm install` runs `prisma generate` for you. Check: `curl http://localhost:4000/health`.

6.3 Frontend

```
cd frontend
pnpm install
pnpm dev
```

Opens on http://localhost:3000SMARTPANTS_PRESERVED_505 (the `pnpm dev` script uses port 3000). API calls go through `lib/axios.ts`.

CONFIDENTIAL

6.4 AI services

```
cd ai-services
npm install
cp .env.example .env
npm run dev
```

Fill `ANTHROPIC_API_KEY`, `OPENROUTER_API_KEY` and a `JWT_SECRET` identical to the backend. Check: `curl http://localhost:5000/health`.

6.5 Daily commands

Task	Command
Stop everything, keep data	`docker compose down`
Start again	`docker compose up -d`
Update to latest code	`git pull` then `docker compose up -d --build`
Follow backend logs	`docker compose logs -f backend`
Open the database UI	`cd backend && npm run db:studio`

Warning: `docker compose down -v` deletes all data (database and files). Use it only to start from zero.

Part 7. Check that the project is healthy

Run these from each workspace.

CONFIDENTIAL

Workspace	Command	What it does
backend	`npm test`	Vitest. Needs `JWT_SECRET` and `JWT_REFRESH_SECRET` (32+ characters) in the environment
backend	`npx vitest run path/to/file.test.js`	One test file
frontend	`pnpm lint`	ESLint including accessibility rules
frontend	`pnpm type-check`	TypeScript strict check
frontend	`pnpm format:check`	Prettier
frontend	`pnpm test`	Vitest
frontend	`pnpm e2e`	Playwright. Needs a running backend with PostgreSQL and Redis
ai-services	`npm test`	Vitest. Needs `JWT_SECRET`

The full CI sequence for local verification is in `docs/CI_LOCAL_VERIFICATION.md`.

CONFIDENTIAL

Part 8. Tour of the project

```
kdl-os/
  backend/          Express API, Prisma schema, modules, seed and scripts
  frontend/         Next.js app: admin panel and public site
  ai-services/      LLM orchestration, RAG, agents
  infra/            Nginx config and helper scripts
  docs/             All guides and references
  agents/           AI agent definitions
  tasks/            Task files
  docker-compose*.yml   Dev, infra-only, prod, staging, e2e and preview stacks
```

8.1 How the backend is built

Every module follows the same chain: routes, then controller, then service, then Prisma.

Rules the code follows everywhere:

- ES Modules only. No CommonJS.
- Zod validates every input.
- Responses use `successResponse` and `errorResponse` from `utils/response.js`.
- Every controller function is wrapped in `try/catch` and calls `next(error)`.
- The Prisma client comes from the `config/database.js` singleton only.
- File uploads go through `storage.service.js`. Nothing writes to disk directly.
- Slow work runs in BullMQ queues, never inline.
- Routes that need permissions use `requirePermission(moduleName, action)` after `authenticate`.
- Every mutation in user management writes an activity log entry.

8.2 The modules

Backend modules live in `backend/src/modules/`.

CONFIDENTIAL

Group	Modules
Core	auth, users, user-management (roles and permissions), settings, setting-fields, identity, storage-settings, media, notifications, integrations, modules
Website content	cms, menus, faq, gallery, team, testimonials, forms, sectors, catalog, work, categories, types, detail-page-types, custom-blocks
Page building	page-builder, page-builder-ui, landing, theme-engine, theme-engine-ui
Studio (Template Engine)	template-engine, brand-kit, collateral, credits, projects, projects-content

Add a module with the generator:

```
cd backend
npm run module:create -- --slug=blog --name="Blog"
```

The anatomy, the `module.json` format and the review checklist are in `docs/MODULE_GUIDE.md`.

8.3 Roles

Role	Can do
`super-admin`	Everything. Bypasses permission checks
`developer`	Builds: all modules
`admin`	Runs content and people
`user`	No admin modules

The defaults live in one file: `backend/src/modules/user-management/shared/role-blueprints.js`.

8.4 The frontend

The Next.js App Router lives in `frontend/src/app/`. The admin panel is under `admin/`, the public site under `(site)/`, sign-in pages under `(auth)/`. Rules:

- TypeScript strict mode.

CONFIDENTIAL

- `useQuery` for reads, `useMutation` for writes.
- Auth state lives in the Zustand store `auth.store.ts`.
- API calls go through `lib/axios.ts`, never raw `fetch`.
- UI changes are made once in a single source, never hardcoded in several places.

CONFIDENTIAL

Part 9. Build a website with the Template Engine

After login, this is the journey an admin follows to build a client website.

Step	Where	What you do
1	Sidebar, Application Settings	Pick the platform. Web Application is the active one
2	Settings, Identity	Enter company name, emails, phones, addresses, logo once. Everything reuses it
3	Sidebar, Modules	Confirm Template Engine, Theme Engine, Page Builder, Menus, Sectors, Catalog, Work and FAQ are on
4	Sidebar, Template Engine	Opens the default project. Click Start Studio run the first time
5	Overview	Upload the logo, check contact details, click Next
6	Color Palette	Four brand colors come from the logo. Keep or edit them, then Submit palette
7	Inference and Approval	AI suggests fonts and tone, a person approves. Brand colors become design tokens
8	Brands, Web app	Typography, font settings, navigation, then Pages. Pages are created and published
9	Layout	Choose Top header, Header and Footer designs
10	Content modules	Fill Sectors, Catalog, Work, FAQ, Case Studies and Team. Each Sector, Catalog and Work entry gets its own detail page
11	Menus	Fine-tune header and footer menus. No re-run needed
12	Edit pages	Visual editor and Section Builder
13	Finish	Guidelines PDF, Collateral, Preflight, Export (all optional)
14	Go live	Open the public pages and re-run only what changed

Public addresses:

CONFIDENTIAL

Address	Shows
`/p/te-<runId>-home` , `-about` , `-contact`	Assembled pages
`/sectors/<slug>`	Sector detail page
`/catalog/<slug>`	Product or service detail page
`/work/<slug>`	Work detail page

What to re-run after a change:

You changed	Do this
Logo or contact details	Save in Overview. Website re-runs by itself if it was already built
Navigation pages	Brands, Web app, Navigation, then Pages
Header, top bar or footer design	Layout, Create layout
Sector, Catalog or Work text and images	Edit the entry in its list. No re-run
Menu order or labels	Sidebar, Menus. No re-run
Brand colors or fonts	Palette or Typography, then Approval, then re-run Website

The permissions needed are `template-engine:view` and `:run` , plus `:approve` for Approval and `:export` for Export. The full screen-by-screen guide is `docs/TEMPLATE_ENGINE_FLOW.md` .

Want more content types such as FAQ, Gallery, Team and Testimonials? Open Modules, find CMS, and click Install.

CONFIDENTIAL

Part 10. Windows setup (once, before Part 3)

1. Click Start, type PowerShell, right-click it, choose Run as administrator.
2. Type `wsl --install` and press Enter. Restart the computer when asked.
3. An Ubuntu window opens. Choose a username and password.
4. Install Docker Desktop. In Docker Desktop go to Settings, Resources, WSL integration, and switch Ubuntu on.
5. In the Ubuntu window type `cd ~` and continue from Part 3.

Open http://localhost/loginSMARTPANTS_PRESERVED_1007 in your normal Windows browser.

Part 11. Production run

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

The production overrides set `restart: always` and `NODE_ENV=production`, and expose only Nginx on port 80. In production the seed refuses to run without `SEED_ADMIN_PASSWORD` (12 or more characters). There is deliberately no default password. Before going live:

- Replace every secret and password in `.env`.
- Set real `CORS_ORIGIN`, `FRONTEND_URL` and `BACKEND_URL` values.
- Keep Nginx SSE settings for `/api/notifications/stream` (`proxy_buffering off`). The shipped `infra/nginx.conf` has them.

CONFIDENTIAL

Part 12. Troubleshooting

Problem	Fix
Error says a variable is missing	Wrong folder, or the `.env` step was skipped. Go to the project folder and repeat Part 4
Port already in use	Close the other program using it, for example a local site on port 80
Build stops with “Killed” or out of memory	Give Docker 8 GB, run `docker compose up -d --build` again
502 Bad Gateway	KDL OS is still starting. Wait 1 or 2 minutes and refresh
Login says wrong password	Did you run the seed? Use `kdl-dev-seed-password` the first time, or run the reset script
429 on login	Rate limit after heavy testing. `docker compose restart backend`
`ECONNREFUSED` on Redis 6380 or `P1001` on PostgreSQL	Infra not running. `docker compose -f docker-compose.infra.yml up -d`
`prisma generate` not found	Run `npm install` inside `backend/`
No logs and no errors from the backend	`mkdir -p backend/logs`
AI chat returns 503	Daily OpenRouter budget used up. It resets at UTC midnight
Spinner forever after refresh	Auth store rehydration. Check the Zustand persist config
Notification stream hangs	Nginx is buffering SSE. See Part 11

CONFIDENTIAL

Part 13. Contributing

1. Read `CLAUDE.md`, `.agents/HANDOFF.md`, `STATUS.md` and `.agents/CONTEXT.md` before starting.
2. `git fetch && git rebase origin/master` before every push. Never push to `master` directly.
3. One concern per pull request. One workspace lockfile per pull request.
4. Run the tests and linters for the workspace you touched. Use exit codes, not impressions.
5. Add your entry to the top of `.agents/HANDOFF.md` and under Rolling changelog in `STATUS.md`.

CONFIDENTIAL

Part 14. Where to read next

Doc	Topic
`docs/INSTALLATION_GUIDE_SIMPLE.md`	Short install guide
`docs/INSTALLATION_GUIDE.md`	Full install guide
`docs/SETUP.md`	Developer onboarding, migrations, webhooks
`docs/ENV_REFERENCE.md`	Every environment variable
`docs/ADMIN_ACCESS.md`	Admin credentials and lockout recovery
`docs/ARCHITECTURE.md`	Folder map and database schema
`docs/API_REFERENCE.md`	All endpoints
`docs/MODULE_GUIDE.md`	How to add a module
`docs/TEMPLATE_ENGINE_FLOW.md`	The website-building journey
`docs/MERGE_DISCIPLINE.md`	Git and pull request rules
`STATUS.md`	What is done and what is pending

CONFIDENTIAL